

EasyFinance API

Справочник разработчика по API v2 мобильного приложения

api.easyfinance.ru/v2

Содержание

1. Введение

2. Аутентификация и авторизация

Регистрация приложения · OAuth · Login/password · Регистрация пользователя · uid · Подпись запроса · Принцип последнего изменения

3. Формат запросов и ответов

GET · POST · Общие параметры · Структура ответа · Удаление объектов

4. Методы API

accounts · operations · categories · tags · targets · budget · calendar · operationPatterns · currencies · systemCategories · users

5. Веб-эндпоинты (сайт)

6. Тарифы и ограничения синхронизации

7. Коды ошибок

8. Чек-лист методов

EasyFinance API — Полная документация разработчика

Настоящий документ — полное описание API `api.easyfinance.ru/v2` мобильного приложения EasyFinance. В нём приведены актуальные правила аутентификации и подписи запросов, все используемые методы с точными параметрами запросов и полями ответов, а также коды ошибок и поведенческие правила синхронизации данных.

1. Введение

- **Base URL:** `https://api.easyfinance.ru/v2/`
- **Формат данных:** JSON. Все ответы обернуты в `{"response": {...}}`.
- **Кодировка:** UTF-8.
- **Формат дат:** строки вида `YYYY-MM-DDTHH:MM:SS+HH:MM` (например, `2026-08-26T15:04:05+03:00`).
- **Типы данных в API:** операции, счета, категории, шаблоны операций, валюты, теги, бюджет пользователя.
- **Полная синхронизация:** за один запрос `users.get` можно получить все данные, но есть ограничение — операции возвращаются только за интервал ± 14 дней от даты запроса. Рекомендуется запрашивать каждый тип данных отдельным методом.

2. Аутентификация и авторизация

2.1. Регистрация приложения (app_id, secret_key)

Каждое стороннее приложение получает при регистрации:

- `app_id` — идентификатор приложения;
- `secret_key` — секретный ключ для расчёта подписи `sig`.

Оба параметра передаются в каждом запросе (см. раздел 3).

2.2. OAuth (authorization code) — используется приложением

Приложение EasyFinance авторизует пользователя по OAuth-флоу:

1. **Получение URL авторизации** — `buildOAuthCodeUrl()` :

```
GET https://api.easyfinance.ru/v2/
    ?app_id=APP_ID
    &response_type=code
    &sig=SIG
```

(без `method` и `access_token`; `sig` строится без `uid`).

Открыть URL в браузере/WebView. Пользователь логинится на сайте и получает `code`.

2. Обмен `code` на `token` — `exchangeCodeForToken(code)` :

```
GET https://api.easyfinance.ru/v2/
?app_id=APP_ID
&code=CODE
&grant_type=authorization_code
&response_type=token
&sig=SIG
```

(без `method` ; `sig` строится **без** `uid`). Сервер возвращает `access_token` и `uid` (в заголовке `location` редиректа или в теле ответа).

3. Далее во всех запросах передаются `access_token` и `uid` (для подписи `sig`).

2.3. Login / password + получение `access_token`

Альтернативный флоу (партнёрские приложения):

- `POST` на сайт `https://easyfinance.ru/login/` с параметрами `login` , `pass` возвращает cookie `PHPSESSID` (используется веб-эндпоинтами, см. раздел 5).
- Для API v2 приложение обменивает связку `app_id` + `secret_key` на `access_token` и `uid` (параметры `app_id` , `secret_key` обмениваются на `access_token` и `uid`).

2.4. Регистрация пользователя (`users.post`)

Нового пользователя можно создать через API методом `users.post` (HTTP POST). В URL — стандартные параметры и подпись, в теле — обязательные поля:

ПОЛЕ	ТИП	ОПИСАНИЕ
<code>login</code>	string	Логин (используется для авторизации)
<code>password</code>	string	Пароль (макс. длина — 40)
<code>name</code>	string	Имя пользователя
<code>mail</code>	string	Электронная почта

Для партнёрских приложений может быть включён режим, при котором сервер сразу возвращает `access_token` в ответе на регистрацию (оговаривается отдельно для каждого приложения, по умолчанию выключен).

2.5. Получение `uid` (`users.get`)

После получения `access_token` идентификатор пользователя `uid` извлекается из ответа `users.get` (поле `id`). `uid` необходим для расчёта подписи всех последующих запросов.

2.6. Подпись запроса (`sig`)

`secret_key` — секрет приложения. Формула (конкатенация строк **без** разделителей, не точка):

- До получения `uid` (обмен `code`→`token`, шаги 2.2): `sig = md5(secret_key + params_string)`

- После получения `uid` (все остальные запросы): `sig = md5(secret_key + uid + params_string)`

Где `params_string` — строка всех параметров запроса (без самого `sig`), значения которых URL-кодируются; порядок параметров важен только для расчёта подписи.

Подробный пример расчёта подписи:

Запрос на получение счётов:

```
https://api.easyfinance.ru/v2/?method=accounts.get&app_id=423004&access_token=be6ef89965d58e56dec21acb9b62bda
```

Параметры для подписи (`params`):

```
method=accounts.get&app_id=423004&access_token=be6ef89965d58e56dec21acb9b62bda
```

Идентификатор пользователя: `uid = 46732644` Секретный ключ: `secret_key = hf78g6f6gf6467f7f77ff7f77f`

Формула:

```
sig = md5( hf78g6f6gf6467f7f77ff7f77f 46732644  
method=accounts.get&app_id=423004&access_token=be6ef89965d58e56dec21acb9b62bd )
```

Результат:

```
sig = b9cc2e5c047d54e96c74adbfa8d43abe
```

Итоговый URL:

```
https://api.easyfinance.ru/v2/?  
method=accounts.get&app_id=423004&access_token=be6ef89965d58e56dec21acb9b62bda&sig=b9cc2e5c047d54e96c74adbfa8d43abe
```

2.7. Принцип последнего изменения

Все изменения данных происходят согласно «последнему изменению». Время фиксируется в поле `updated_at`, которое есть у каждого объекта (кроме тегов). Сервер сравнивает `updated_at` присланных данных с данными на сервере — у кого `updated_at` больше, те и приоритетнее. Если пользователь менял данные в приложении, но не синхронизировал их, а затем изменил их же на сайте, то при последующей синхронизации приоритет останется у серверной версии.

3. Формат запросов и ответов

3.1. GET

Параметры в query-строке: `method` , `app_id` , `access_token` , <доп. параметры> , `sig` . Все они (включая дополнительные) входят в `params_string` для подписи.

```
GET https://api.easyfinance.ru/v2/
?method=accounts.get
&app_id=APP_ID
&access_token=TOKEN
&fields=id,name
&sig=SIG
```

3.2. POST

- Параметры в query: `method` , `app_id` , `access_token` , `transact_key` , <id-параметры: `operation_id` / `chain_id` / `category_id` / `tag_id` / `target_id` / `operation_pattern_id`> (все входят в подпись), `sig` .
- Тело запроса — JSON в `Content-Type: application/json` :

```
{
  "request": {
    "request_data": { "<объект или массив записей>" }
  }
}
```

Конкретные обёртки (`operations` , `accounts` , `categories` , `tags` , `targets` , `operationPatterns` , `calendar` , `budgets`) указаны в каждом методе.

3.3. Общие параметры

- `fields` — список возвращаемых полей через запятую (по умолчанию — все). Для счётов/операций через `fields` можно запросить `init_balance`,`balance` .
- `options` — опции: `client` (вернуть `client_id`), `noresponse` (не возвращать тело). Для POST передаётся в query.
- `transact_key` — обязателен для всех POST; уникальный ключ идемпотентности, сгенерированный клиентом (при повторе с тем же ключом сервер не создаёт дубль).
- `client_id` — идентификатор объекта в стороннем приложении (используется для сопоставления при создании счетов/операций/категорий/шаблонов).
- `account_list` / `operation_list` / `category_list` / `tags_list` / `operation_pattern_list` — списки id через запятую для выборочной выгрузки.

3.4. Структура ответа

Успех:

```
{ "response": { "response_data": { "accounts": [ ... ] } } }
```

Ошибка (вложенная):

```
{ "response": { "response_error": { "error_code": "...", "error_message": "..." } } }
```

Или список ошибок валидации:

```
{ "response": { "response_data": { "errors": [ { "code": ..., "text": "..." } ] } } }
```

3.5. Удаление объектов

У большинства объектов **нет** выделенного метода `*.delete` — удаление выполняется через соответствующий `*.set` с поставленным полем `deleted_at`, равным времени удаления (исключение — счета, где также используется `state = "2"`):

```
deleted_at = 2012-12-12T00:57:58+0200
```

Сервер делает soft-delete. **Исключение:** у календаря/планировщика есть собственный метод `calendar.delete` (см. §4.7) — для удаления запланированного события им и следует пользоваться.

4. Методы API

4.1. accounts.* — счета

accounts.get

GET. Параметры: `account_list` (ids, опц.), `fields` (опц., напр. `init_balance, balance`). **Ответ:** `response_data.accounts[ ]`. Поля счёта:

ПОЛЕ	ТИП	ОПИСАНИЕ
<code>id</code>	int	Идентификатор счёта
<code>name</code> / <code>title</code>	string	Название
<code>balance</code>	decimal	Текущий баланс
<code>init_balance</code>	decimal	Начальный баланс
<code>currency_id</code>	int	ID валюты (1=RUB, 2=USD, 3=EUR, 4=GBP, 5=CHF, 6=CNY, 7=JPY, 8=BYN, 9=UAH, 10=KZT, 11=PLN, 12=CZK, 13=SEK, 14=NOK)

currency_char_code	string	Код валюты (RUB, USD, ...)
type_id	int	Тип счёта (см. маппинг ниже)
state	int	1 — избранный, 2 — в архиве
icon	string	Код иконки accountimageN
include_in_total	0/1	Учитывать в общем балансе
created_at , updated_at	datetime	Даты
description	string	Описание (кредиты)
bank_id	int	ID банка (кредиты)
annual_rate	decimal	Годовая ставка (кредиты)
payment_type	string	Тип платежа (кредиты)
open_date , close_date	date	Даты (кредиты)
commission_one_time , commission_monthly	decimal	Комиссии (кредиты)
payment_day	int	День платежа (кредиты)
credit_limit	decimal	Кредитный лимит (кредиты)

Маппинг type_id → тип: 1 cash, 2 card, 5 deposit, 6 loan_given, 7 loan_received, 8 credit_card, 9 credit, 10 oms, 11 stocks, 12 pif, 13 ofbu, 14 pension, 15 electronic, 16 bank_account, 17 real_estate, 18 car, 19 other_securities, 20 fund, 21 insurance_savings, 22 savings_plan, 23 npf, 24 water_transport, 25 art, 26 business, 27 other_property, 28 air_transport, 29 motorcycle, 30 bonds, 31 pamm, 32 broker, 33 bonus_card.

Иконки: accountimage1 cash, accountimage2 credit_card, accountimage3 savings, accountimage4 account_balance, accountimage5 wallet, accountimage6 payments, accountimage7 currency_ruble, accountimage8 card_giftcard.

accounts.post

POST. Query: transact_key (и опц. options=client). **Тело:** {"request":{"request_data":{"accounts":[{" ... }]}}} . Поля записи: name , init_balance (строка, 2 знака), type_id , state ("0"), currency_id ("1" если не задан), icon (accountimageN), include_in_total ("1" / "0"), created_at , updated_at ; для кредитных — опц. description , bank_id , annual_rate , payment_type , open_date , close_date , commission_one_time , commission_monthly , payment_day , credit_limit . **Ответ:** response_data.accounts[0].id .

accounts.set

POST. Query: transact_key , account_id . **Тело:** {"request":{"request_data":{"accounts":[{"id": "...", те же поля, что у post, + "updated_at" }]}}} . Используется для редактирования, отметки «избранный» (state="1") и архивации/удаления (state="2" , либо deleted_at). Одним запросом — любое количество счетов.

4.2. operations.* — операции

operations.get

GET. Получение списка операций пользователя.

Параметры (все опциональные):

ПАРАМЕТР	ТИП	ОПИСАНИЕ
operation_list	string	Списки id операций через запятую (для выборочной выгрузки)
from	string	Начальная дата периода. Формат ISO 8601 : YYYY-MM-DDTHH:MM:SS+HH:MM
to	string	Конечная дата периода. Формат ISO 8601 : YYYY-MM-DDTHH:MM:SS+HH:MM
interval_field	string	Какое поле даты фильтровать. Допустимые значения: date (дата операции), created_at , updated_at , deleted_at
fields	string	Список полей через запятую (по умолчанию — все)
options	string	Опции: client (вернуть client_id), init_balance,balance (эмуляция журнала балансов)

Важные правила:

- Параметры from и to передаются **вместе** — если задан только один, сервер вернёт ошибку
- interval_field обязателен при использовании from / to
- Формат даты строго ISO 8601 с таймзоной (например, 2026-09-01T00:00:00+03:00)
- Если from / to не заданы — сервер возвращает **все** операции пользователя
- Валидация from / to выполняется в BasicRequestForm.class.php через валидатор myValidatorDatetimeIso8601

Пример запроса (операции за последние 3 месяца):

```
GET https://api.easyfinance.ru/v2/
?method=operations.get
&app_id=APP_ID
&access_token=TOKEN
&from=2026-06-01T00:00:00+03:00
&to=2026-09-04T23:59:59+03:00
&interval_field=date
&sig=SIG
```

Пример запроса (все операции, без фильтрации):

```
GET https://api.easyfinance.ru/v2/
?method=operations.get
&app_id=APP_ID
&access_token=TOKEN
&sig=SIG
```

Ответ: response_data.operations[] . Поля операции:

ПОЛЕ	ТИП	ОПИСАНИЕ
id	int	Идентификатор операции
account_id	int	ID счёта
amount	decimal	Сумма (положительное; знак определяется type)
date	string	Дата операции (YYYY-MM-DD)
time	string	Время операции (HH:MM:SS)
category_id	int	ID категории (у перевода — служебная категория «Перевод»)
comment	string	Комментарий
accepted	0/1	Подтверждена ли операция
tags	string	Теги через запятую или JSON-массив
type	int	0 — расход, 1 — доход, 2 — перевод
transfer_account_id	int	ID счёта получателя (для перевода)
transfer_amount	decimal	Сумма перевода на счёт получателя
created_at	string	Дата/время создания (ISO 8601)
updated_at	string	Дата/время обновления (ISO 8601)
deleted_at	string	Дата/время удаления (ISO 8601, soft-delete)
client_id	string	ID объекта в стороннем приложении (через options=client)
mcc_code	string	MCC-код мерчанта
merchant_name	string	Название мерчанта

Структура ответа (пример):

```
{
  "response": {
    "response_data": {
      "operations": [
        {
          "id": "123456",
          "account_id": "789",
          "amount": "1500.00",
          "date": "2026-09-01",
          "time": "14:30:00",
          "category_id": "456",
          "comment": "Покупка",
          "accepted": 1,
          "tags": "еда,магазин",
          "type": 0,
          "transfer_account_id": null,
          "transfer_amount": null,
          "created_at": "2026-09-01T14:30:00+03:00",
          "updated_at": "2026-09-01T14:30:00+03:00"
        }
      ]
    }
  }
}
```

```
}  
}  
}
```

Обработка ошибок валидации:

```
{  
  "response": {  
    "response_data": {  
      "errors": [  
        {"code": 46, "text": "Invalid field 'from'"},  
        {"code": 48, "text": "Invalid field 'to'"}  
      ]  
    }  
  }  
}
```

Коды ошибок для `operations.get` :

КОД	ЗНАЧЕНИЕ
46 (<code>invalid_field_from</code>)	Неверный формат параметра <code>from</code>
48 (<code>invalid_field_to</code>)	Неверный формат параметра <code>to</code>

`operations.post`

POST. Query: `transact_key` , options=`client` . Тело: `{"request":{"request_data": {"operations":[{" ... }]]}}` . Поля записи:

- `type` — 0 / 1 / 2 .
- `user_id` — id пользователя.
- `account_id` — счёт списания.
- `category_id` — категория (для перевода — служебная категория «Перевод»).
- `currency_id` — валюта счёта.
- `amount` — **знаковое** число: расход и перевод — отрицательное, доход — положительное, формат с 2 знаками.
- `date` (`YYYY-MM-DDTHH:MM:SS+HH:MM`), `time` (`HH:MM:SS`).
- `transfer_account_id` , `transfer_amount` — для перевода (`transfer_amount` положительное).
- `comment` , `tags` (опц.).
- `accepted` — `true` .
- `client_id` — для сопоставления с серверным `id` .
- `created_at` , `updated_at` .

ВАЖНО: тип категории операции должен соответствовать типу операции. У расхода — расходная категория, у дохода — доходная, у перевода — категория «Перевод» (её приложение добавляет автоматически, пользователю не показывает).

operations.set

POST. Query: `transact_key` , `operation_id` . **Тело:** `{"request":{"request_data":{"operations":[{"id": "...", те же поля, что у post, + "updated_at", "deleted_at": null }]]}}` . Для удаления — `deleted_at` (либо `state="2"`). У перевода категорию нельзя изменить. Одним запросом — любое количество операций.

4.3. categories.* — категории

categories.get

GET. Параметры (опц.): `category_list` , `fields` . **Ответ:** `response_data.categories[]` . Поля: `id` , `system_id` , `name` , `type` (`-1` расход, `1` доход), `custom` (`0` системная, `1` пользовательская), `icon` (`catimgN` , `catimg1..catimg33`), `parent_id` , `is_hidden` (`0 / 1`), `created_at` , `updated_at` , `deleted_at` , `client_id` .

categories.post

POST. Query: `transact_key` . **Тело:** `{"request":{"request_data":{"categories":[{"name", "type":"-1"| "1", "icon":"catimgN", "system_id":"0", "custom":"1", "parent_id":"0", "is_hidden":"0", "created_at", "updated_at" }]]}}` . При создании новой категории системная категория должна соответствовать её типу (расход → системные только с `type=-1`). **Ответ:** `response_data.categories[0].id` .

categories.set

POST. Query: `transact_key` , `category_id` . **Тело:** `{"request":{"request_data":{"categories":[{"id", "name", "type", "icon":"catimgN", "system_id", "custom":"0"| "1", "parent_id", "is_hidden":"0", "created_at", "updated_at", "deleted_at" (для удаления) }]]}}` . Удаление — через `deleted_at` . Одним запросом — любое количество категорий.

***Системная категория «Перевод»** (`is_public=0`) — добавляется ко всем операциям перевода, но не показывается пользователю.*

4.4. tags.* — теги

tags.get

GET. Параметры (опц.): `fields` , `tags_list` . **Ответ:** `response_data.tags[]` . Поля тега: `id` , `user_id` , `text` (текст тега — **поле называется text, не name**), `operation_id` (опц.).

tags.post

POST. Query: `transact_key` . **Тело:** `{"request":{"request_data":{"tags":[{"name": "<текст>" }]]}}` . При **записи** поле — `name` , при **чтении** сервер возвращает `text` . **Ответ:** созданный тег с `id` .

tags.set

POST. Query: `transact_key` , `tag_id` . **Тело:** `{"request":{"request_data":{"tags":[{"id", "name", "deleted_at":"<дата>" }]]}}` . Используется для «мягкого» удаления тега (проставить `deleted_at`). Отдельного `tags.delete` нет.

4.5. targets.* — цели

targets.get

GET. Ответ: `response_data.targets[]` . Поля: `id` , `title` , `type` (0), `state` (0), `amount` (цель, строкой), `amount_done` (накоплено, строкой), `currency_id` (1 RUB), `account_id` , `category_id` , `date_begin` , `date_end` , `comment` , `photo` , `url` , `visible` (1 / 0), `close` , `done` . Сервер возвращает ВСЕ цели, включая `visible=0` ; клиент фильтрует скрытые сам.

targets.post

POST. Query: `transact_key` , `options=client` . **Тело:** `{"request":{"request_data":{"targets":[{"title", "amount", "amount_done":"0.00", "end":"YYYY-MM-DD" (необяз.), "currency_id":"1", "date_begin", "date_end", "account_id", "category_id", "comment", "visible":"1" }]]}}` . **Ответ:** созданная цель с `id` .

targets.set

POST. Query: `transact_key` , `target_id` . **Тело:** `{"request":{"request_data":{"targets":[{"id": "<target_id>", "...поля цели }]]}}` . Для скрытия/удаления достаточно `{"id": "<target_id>", "visible": "0"}` .

4.6. budget.* — бюджет

budget.get

GET. Ответ: `response_data.budget` (объект, не массив). Поля: `planned` (плановая сумма на период), `spent` (фактически потрачено), `date_start` , `date_end` .

budget.categoriesget (имя метода — слитно, без точки)

GET. Ответ: `response_data.budgets[]` . Поля: `id` , `category_id` (к какой категории план), `planned` (план на период, строкой), `spent` (фактически потрачено за период).

budget.categoriespost

POST. Query: `transact_key` , `options=client` . **Тело:** `{"request":{"request_data":{"category_id": "<id>", "planned": "<сумма>" }}}` .

budget.categoriesset

POST. Query: `transact_key` . **Тело:** `{"request":{"request_data":{"id": "<id плана>", "category_id": "<id>", "planned": "<сумма>" }}}` .

4.7. calendar.* — календарь / планировщик

calendar.get

GET. Параметры (опц.): `from` , `to` , `options` (`accepted` — только подтверждённые вхождения). **Ответ:** `response_data.calendar[]` . Поля события: `id` , `operation_id` , `chain_id` , `account_id` , `transfer_account_id` , `category_id` , `amount` , `date` , `time` , `comment` , `type` (`0` расход, `1` доход, `2` перевод), `accepted` , `every_day` (`1` / `7` / `30` / `90` / `365`), `repeat` (`"1"` разовый, `"0"` до `date_end` , иначе — число вхождений), `date_start` , `date_end` , `week_days` (маска 7 символов Пн..Вс), `tags` .

calendar.post

POST. Query: `transact_key` . Тело: `{"request":{"request_data":{"<объект события> }}}` (без массива). Поля: `account_id` , `category_id` , `amount` , `date` , `time` , `comment` , `type` , `transfer_account_id` , `transfer_amount` , `accepted` , `every_day` , `date_start` , `date_end` , `repeat` , `week_days` . **Ответ:** созданное событие с `id` и `chain_id` .

calendar.set

POST. Query: `transact_key` , `operation_id` , `chain_id` . Тело: `{"request":{"request_data":{"<объект события> }}}` .

calendar.delete

POST. Query: `transact_key` , `operation_id` , `chain_id` . Тело: `{"request":{"request_data":{}}}` .

calendar.accept

POST. Query: `transact_key` , `operation_id` , `chain_id` . Тело: `{"request":{"request_data":{"date":"YYYY-MM-DD", "accepted":1}}}` .

4.8. operationPatterns.* — шаблоны операций

Шаблоны операций доступны только для стороннего приложения (на сайте их нет).

operationPatterns.get

GET. Параметры (опц.): `operation_pattern_list` , `fields` . **Ответ:** `response_data.operationPatterns[]` . Поля: `id` , `type` (`0` расход, `1` доход, `2` перевод, `3` начальный остаток, `4` перевод на цель), `account_id` , `transfer_account_id` , `transfer_amount` , `category_id` , `user_id` , `amount` , `name` , `icon` (`{ "code": "catimgN" }`), `comment` , `tags` , `created_at` , `updated_at` , `deleted_at` , `client_id` .

operationPatterns.post

POST. Query: `transact_key` , `options=client` . Тело: `{"request":{"request_data":{"operationPatterns":[{"client_id", "user_id", "name", "type" (int), "icon":`

`{"code":"catimgN"}, "amount", "account_id" (опц.), "category_id" (опц.), "transfer_account_id" (опц.), "comment" (опц.), "tags" (опц.), "created_at", "updated_at" }]]]]` . **Ответ:** `response_data.operationPatterns[0].id` .

`operationPatterns.set`

POST. Query: `transact_key` , `operation_pattern_id` . **Тело:** `{"request":{"request_data":{"operationPatterns":[{"id", "deleted_at":"<дата>" (для удаления) | полный рекорд для редактирования }]]}}` . Удаление шаблона — через `deleted_at` .

4.9. `currencies.get` — валюты

GET. Ответ: `response_data.currencies[]` . Поля: `id` , `char_code` (например, `RUB`), `currency_char_code` , `name` , `rate` (опц.), `symbol` (опц.).

4.10. `systemCategories.get` — системные категории

GET. Ответ: `response_data.systemCategories[]` — те же поля, что у `categories.get` (встроенные системные категории, `custom=0`). Используются как основа при создании пользовательских категорий (тип должен совпадать).

4.11. `users.get` — данные пользователя

GET. Параметры (опц.): `fields` (например, `goals`) . **Ответ:** поля на уровне `response_data` : `id` , `name / title` , `login` , `mail / email` , `account_type` , `default_currency` (`id` валюты → код), `tariff_duration` (дата окончания тарифа), `created_at` . При `fields=goals` дополнительно `response_data.goals[]` — список целей пользователя.

Для полной синхронизации всех данных за один запрос используйте `users.get` с параметром `fields` , перечисляющим нужные типы данных.

5. Веб-эндпоинты (сайт)

Не используют схему `method=` и требуют cookie `PHPSESSID` (выдаётся при веб-логине, см. раздел 2.3).

- **Веб-логин:** `POST https://easyfinance.ru/login/` с телом `login` , `pass` → cookie `PHPSESSID` .
- **События календаря (сайт):** `GET https://easyfinance.ru/calendar/events/?responseMode=json`
- **Создание события календаря (сайт):** `POST https://easyfinance.ru/calendar/add/?responseMode=json` (form-urlencoded)
- **Удаление события календаря (сайт):** `POST https://easyfinance.ru/calendar/delete/?responseMode=json` (form-urlencoded: `id` , `chain`)
- **Обратная связь:** `POST https://easyfinance.ru/feedback/add_message/?responseMode=json` (form-urlencoded: `title` , `msg` , `email`)

6. Тарифы и ограничения синхронизации

На сервисе EasyFinance.ru действует тарифная система; набор доступных возможностей зависит от тарифа пользователя. Синхронизация с мобильным приложением доступна, в том числе на бесплатном тарифе. Тариф пользователя определяется по полю `tariff_duration` в ответе `users.get` (см. раздел 4.11): если дата окончания тарифа задана и ещё не наступила, пользователь считается платным (`User.isPremium == true`), иначе — бесплатным.

Ограничение количества операций. На бесплатном тарифе доступно хранение до 1000 операций. Проверка выполняется на стороне клиента приложения (`FinanceStore.addOperation`): когда число загруженных операций достигает 1000, попытка добавить следующую (1001-ю) операцию блокируется **до обращения к API**, и пользователю показывается диалог:

- заголовок: «Лимит операций»;
- текст: «Достигнут лимит в 1000 операций. Обновите тарифный план, чтобы добавить больше.»;
- кнопка «Обновить тариф», которая открывает страницу оплаты `https://easyfinance.ru/my/shop` (переход выполняется средствами `url_launcher` — ссылка открывается во внешнем браузере).

У пользователей с активным платным тарифом (`isPremium == true`) это ограничение не применяется — они могут добавлять операции без лимита в 1000 штук. Сам лимит (1000) жёстко задан в коде; если сервер накладывает собственные ограничения для платных тарифов, они проверяются отдельно на стороне API.

В объекте пользователя (`users.get`) есть поле `tariff_duration` — дата окончания тарифа; приложение должно информировать пользователя о его окончании.

7. Коды ошибок

Ошибки валидации возвращаются в `response_data.errors[]` (или в `response_error`). Ниже — основные коды ошибок.

Пользователь (user)

КОД	ЗНАЧЕНИЕ
<code>user_invalid_login</code>	Неверный логин
<code>user_required_login</code>	Логин обязателен
<code>user_invalid_name</code>	Неверное имя
<code>user_required_name</code>	Имя обязательно
<code>user_invalid_user_mail</code>	Неверный email
<code>user_required_user_mail</code>	Email обязателен
<code>user_invalid_password</code>	Неверный пароль (макс. длина — 40)
<code>user_required_password</code>	Пароль обязателен

Счета (account)

КОД	ЗНАЧЕНИЕ
account_invalid_type_id	Неверный type_id
account_required_type_id	type_id обязателен
account_invalid_currency_id	Неверный currency_id
account_required_currency_id	currency_id обязателен
account_invalid_name	Неверное имя
account_required_name	Имя обязательно
account_invalid_description	Неверное описание
account_required_description	Описание обязательно
account_invalid_init_balance	Неверный init_balance
account_required_init_balance	init_balance обязателен
account_invalid_state	Неверный state
account_required_state	state обязателен
account_invalid_created_at	Неверный created_at
account_required_created_at	created_at обязателен
account_invalid_updated_at	Неверный updated_at
account_required_updated_at	updated_at обязателен
account_invalid_deleted_at	Неверный deleted_at
account_required_deleted_at	deleted_at обязателен
account_required_id	id обязателен

Категории (category)

КОД	ЗНАЧЕНИЕ
category_invalid_system_id	Неверный system_id
category_required_system_id	system_id обязателен
category_invalid_parent_id	Неверный parent_id
category_required_parent_id	parent_id обязателен
category_invalid_name	Неверное имя
category_required_name	Имя обязательно
category_invalid_type	Неверный type
category_required_type	type обязателен

category_invalid_created_at	Неверный created_at
category_required_created_at	created_at обязателен
category_invalid_updated_at	Неверный updated_at
category_required_updated_at	updated_at обязателен
category_invalid_deleted_at	Неверный deleted_at
category_required_deleted_at	deleted_at обязателен

Операции (operation)

КОД	ЗНАЧЕНИЕ
operation_invalid_amount	Неверная сумма
operation_required_amount	Сумма обязательна
operation_invalid_date	Неверная дата
operation_required_date	Дата обязательна
operation_invalid_time	Неверное время
operation_required_time	Время обязательно
operation_invalid_type	Неверный тип
operation_required_type	Тип обязателен
operation_invalid_comment	Неверный комментарий
operation_required_comment	Комментарий обязателен
operation_invalid_tags	Неверные теги
operation_required_tags	Теги обязательны
operation_invalid_acceptedt	Неверный accepted (опечатка в API)
operation_required_accepted	accepted обязателен
operation_invalid_transfer_account_id	Неверный transfer_account_id
operation_required_transfer_account_id	transfer_account_id обязателен
operation_invalid_transfer_amountt	Неверный transfer_amount (опечатка в API)
operation_required_transfer_amount	transfer_amount обязателен
operation_invalid_created_at	Неверный created_at
operation_required_created_at	created_at обязателен
operation_invalid_updated_at	Неверный updated_at
operation_required_updated_at	updated_at обязателен
operation_invalid_deleted_at	Неверный deleted_at

<code>operation_required_deleted_at</code>	deleted_at обязателен
<code>operation_invalid_user_id</code>	Неверный user_id
<code>operation_required_user_id</code>	user_id обязателен
<code>operation_invalid_id</code>	Неверный id
<code>operation_required_id</code>	id обязателен
46 (<code>invalid_field_from</code>)	Неверный формат параметра <code>from</code> (ожидается ISO 8601)
48 (<code>invalid_field_to</code>)	Неверный формат параметра <code>to</code> (ожидается ISO 8601)

Шаблоны операций (operationPattern)

КОД	ЗНАЧЕНИЕ
<code>operationPattern_invalid_type</code>	Неверный тип
<code>operationPattern_invalid_deleted_at</code>	Неверный deleted_at
<code>operationPattern_required_deleted_at</code>	deleted_at обязателен

Общие

КОД	ЗНАЧЕНИЕ
<code>account_have_operations</code>	Счёт нельзя удалить — есть операции
<code>category_have_operations</code>	Категорию нельзя удалить — есть операции

8. Чек-лист методов

Методы, реально вызываемые приложением (полный список): `accounts.get` , `accounts.post` , `accounts.set` , `operations.get` , `operations.post` , `operations.set` , `categories.get` , `categories.post` , `categories.set` , `tags.get` , `tags.post` , `tags.set` , `targets.get` , `targets.post` , `targets.set` , `budget.get` , `budget.categoriesget` , `budget.categoriespost` , `budget.categoriesset` , `calendar.get` , `calendar.post` , `calendar.set` , `calendar.delete` , `calendar.accept` , `operationPatterns.get` , `operationPatterns.post` , `operationPatterns.set` , `currencies.get` , `systemCategories.get` , `users.get` . А также OAuth (`buildOAuthCodeUrl` , `exchangeCodeForToken`) и веб-эндпоинты (раздел 5).

Выделенного метода `*.delete` нет у большинства объектов — удаление

`accounts/categories/tags/operationPatterns/operations` выполняется через `*.set` с `deleted_at` (или `state="2"` для счетов). Исключение: у календаря есть собственный метод `calendar.delete` (§4.7).